

Autonomous Campus Tour Robot

Final Design Report

Course	ENGR 122 - Design of Systems
Semester	Spring 2026
Section	C1
Group Number	G8
Team Members and Majors	Cristiano Ambrosio – Civil Engineering
	Anthony Grigoli – Mechanical Engineering
	Alex Santora – Computer Engineering
Instructor	Prof. Chang Beom Joo
Submission Date	2026-04-4

Stevens Honor Pledge: I pledge my honor that I have abided by the Stevens Honor System

Signatures: Cristiano Ambrosio, Anthony Grigoli, Alex Santora

Table of Contents

1. Executive Summary
 2. Introduction
 3. Project Objectives and Design Requirements
 4. Concept Design and Design Alternatives
 5. Final Design Description
 6. Assembly and Prototyping
 7. Performance Reporting and Project Management
 8. Conclusion and Recommendation
- Appendix A: Full Arduino Sketch
- Appendix B: Supporting Files Checklist

1. Executive Summary

The purpose of this project was to design, build, program, and test an autonomous driving robot that can guide visitors through a Stevens Institute of Technology campus-tour arena. The final design is intended to move through a 96 in by 36 in arena, receive robot position data through the MQTT network, use camera-based ArUco marker tracking for current x-y coordinates, avoid five campus-building obstacles with ultrasonic sensors, and reach four client-provided target locations in the required order. The design was created for a six-week development window and was limited by course materials, 3D-printing time, safety requirements, and final competition rules.

The final robot design uses a WeMos D1 R1 as the main controller, a motor driver for differential-drive movement, at least three HC-SR04 ultrasonic sensors for obstacle detection, an OLED display for live debugging feedback, a custom power bus for stable power distribution, and an ArUco marker bracket that supports two 4 cm by 4 cm markers. The mechanical design focuses on keeping the marker bracket stable and visible to the overhead camera while maintaining a low center of gravity and clear sensor fields of view. The electrical design removes the breadboard from the final system and uses organized power and ground branches with a slide switch on the positive battery line.

The software design combines target reaching and obstacle avoidance. During operation, the robot connects to Wi-Fi and MQTT, receives its current x-y coordinates, reads ultrasonic sensor distances, computes the error between the current position and the active target, and sends motor commands that move the robot toward the target while avoiding obstacles. When the robot enters the 10 cm target radius, it stops for 2 seconds before advancing to the next target. This satisfies the main performance requirements of autonomous path planning, collision avoidance, and target confirmation.

The main design innovations are the integrated ArUco marker bracket, the organized no-breadboard power bus, and the use of live OLED status messages to make debugging easier during competition testing. Future improvements should focus on tuning the path-planning algorithm, adding stronger sensor filtering, and improving the mechanical sensor mounts for more repeatable distance measurements.

2. Introduction

The final project required the team to develop an autonomous robot for a simulated Stevens campus tour. The robot had to move safely through an arena, reach target coordinates, and avoid obstacles representing Stevens campus buildings. The system combined mechanical design, electrical wiring, embedded programming, and project management. Because the robot had to respond to coordinate data and obstacles during operation, the project required both a physical robot design and a software control strategy.

The main technologies used in the final design include a WeMos D1 R1 microcontroller, MQTT communication over Wi-Fi, an overhead camera system that tracks ArUco markers, HC-SR04 ultrasonic sensors, an OLED display, a motor controller board, two DC drive motors, a battery pack, a slide switch, and a 3D-printed robot body. The Smart Arena provides the robot location through x-y coordinate data. The robot software uses that data to determine the direction and distance to the next target.

The arena includes five fixed obstacles representing Babbio Center, Gate House, The Torch Bearers, Edwin A. Stevens Hall, and Wesley J. Howe Center. Since the target locations may change on the day of competition, the robot cannot rely only on pre-programmed timing. Instead, the robot must use its live coordinate position and sensor readings to make decisions.

3. Project Objectives and Design Requirements

The overall design objective was to create an autonomous robot that could guide visitors to a sequence of target locations without collisions. A successful design needed to be accurate, stable, safe, and easy to maintain. The objectives and requirements were separated into performance, mechanical, electrical, software, and documentation categories.

Category	Requirement	Reason	Verification Method
Performance	Reach four target locations in the client-specified order.	Shows the robot can complete the campus-tour mission.	Arena trial results and target log.
Performance	Stop for 2 seconds at the start and after each reached target.	Confirms arrival and follows competition rules.	Timer observation and OLED status.
Performance	Reach targets within a 10 cm radius.	Defines target accuracy.	Compare final x-y coordinates to target coordinates.
Safety	Avoid collisions with obstacles and arena walls.	Protects the robot and proves obstacle avoidance.	Count crashes during testing.
Mechanical	Use a bracket supporting two 4 cm by 4 cm ArUco markers.	Allows the camera system to track the robot.	Visual inspection and camera detection test.
Mechanical	Keep the top of the ArUco marker below 6.5 in from the ground.	Satisfies height limit and improves detection reliability.	Measure bracket height.
Mechanical	Do not use glue or tape to secure parts.	Improves reliability and follows project rules.	Assembly inspection.
Electrical	Do not use a breadboard in the final robot.	Prevents loose connections during motion.	Final wiring inspection.
Electrical	Include a power bus and a slide switch on the positive battery line.	Distributes power safely and allows full shutoff.	Continuity test and power-on test.
Electrical	Follow color coding for power and ground wiring.	Reduces wiring errors and troubleshooting time.	Visual inspection.
Software	Use MQTT coordinate data for path planning.	Prevents timing-only control and allows changing targets.	Serial/OLED coordinate display and route test.
Software	Use at least three ultrasonic sensors in path-planning decisions.	Allows front and side obstacle detection.	Sensor reading log and avoidance behavior.
Documentation	Submit final report, Arduino sketch, SolidWorks files, and supporting documents.	Allows the client/instructor to review and maintain the design.	Submission checklist.

The most important constraint was that the robot had to operate autonomously using sensor feedback rather than remote control. Another major constraint was time. The team had six weeks to update the body design, build the circuit, write and test the code, prepare the report, and participate in the Week 13 competition. The 3D-printing time limit also affected concept selection because large or complicated parts could exceed the available printing window.

4. Concept Design and Design Alternatives

Several design concepts were considered before selecting the final design. The main choices involved the ArUco marker bracket shape, ultrasonic sensor placement, power distribution method, and chassis layout. The selected design favored stability, printability, easy wiring access, clear sensor views, and a low center of gravity.

Concept	Description	Advantages	Disadvantages	Decision
A: Tall vertical marker tower	A tall post mounted near the rear of the robot to hold two ArUco markers.	Easy camera visibility.	Higher center of gravity and possible 6.5 in height violation.	Rejected.
B: Low dual-marker bridge	A lower bracket that holds two markers	Stable, visible, easier to print, lower tipping	Requires careful mounting so wires do	Selected.

	above the robot but below the height limit.	risk.	not block markers.	
C: Front-only ultrasonic layout	Three sensors focused mostly forward.	Simple wiring and easy code.	Weak side detection when turning near obstacles.	Modified.
D: Front-left-right ultrasonic layout	Sensors aimed forward, left, and right.	Balanced obstacle detection for path correction.	Slightly more complicated mounting and logic.	Selected.
E: Breadboard-based wiring	Use breadboard for final distribution.	Fast prototyping.	Not allowed in final design and unreliable under vibration.	Rejected.
F: Wire power bus or soldered PCB	Use spliced branches or a soldered board for power distribution.	More reliable, compact, and rule-compliant.	Requires planning and continuity testing.	Selected.

The priority check method favored the low dual-marker bridge and front-left-right sensor arrangement. Print time was a major factor in rejecting larger brackets. Sensor placement was selected so the robot could detect objects directly ahead and on both sides while still leaving enough space for the OLED display, wiring, and motor controller board.

Insert hand sketches or screenshots of early design alternatives here. Label each figure clearly, such as Figure 1: ArUco tower concept and Figure 2: Low bridge concept.

5. Final Design Description

5.1 System Description and Architecture

The final system is organized around a main controller that receives input data, processes decisions, and sends output commands. Inputs include MQTT x-y coordinate data from the Smart Arena camera system and ultrasonic distance readings. Outputs include motor commands to the motor driver and status messages to the OLED display. The power subsystem supplies the controller, sensors, display, and motor driver through a power bus controlled by a slide switch.

System Architecture: Inputs, Processing, Outputs, and Power

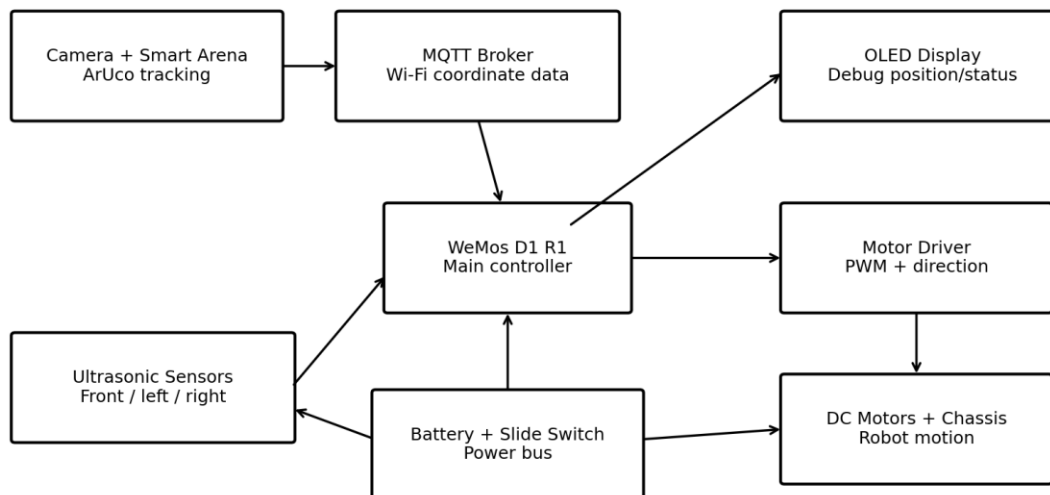


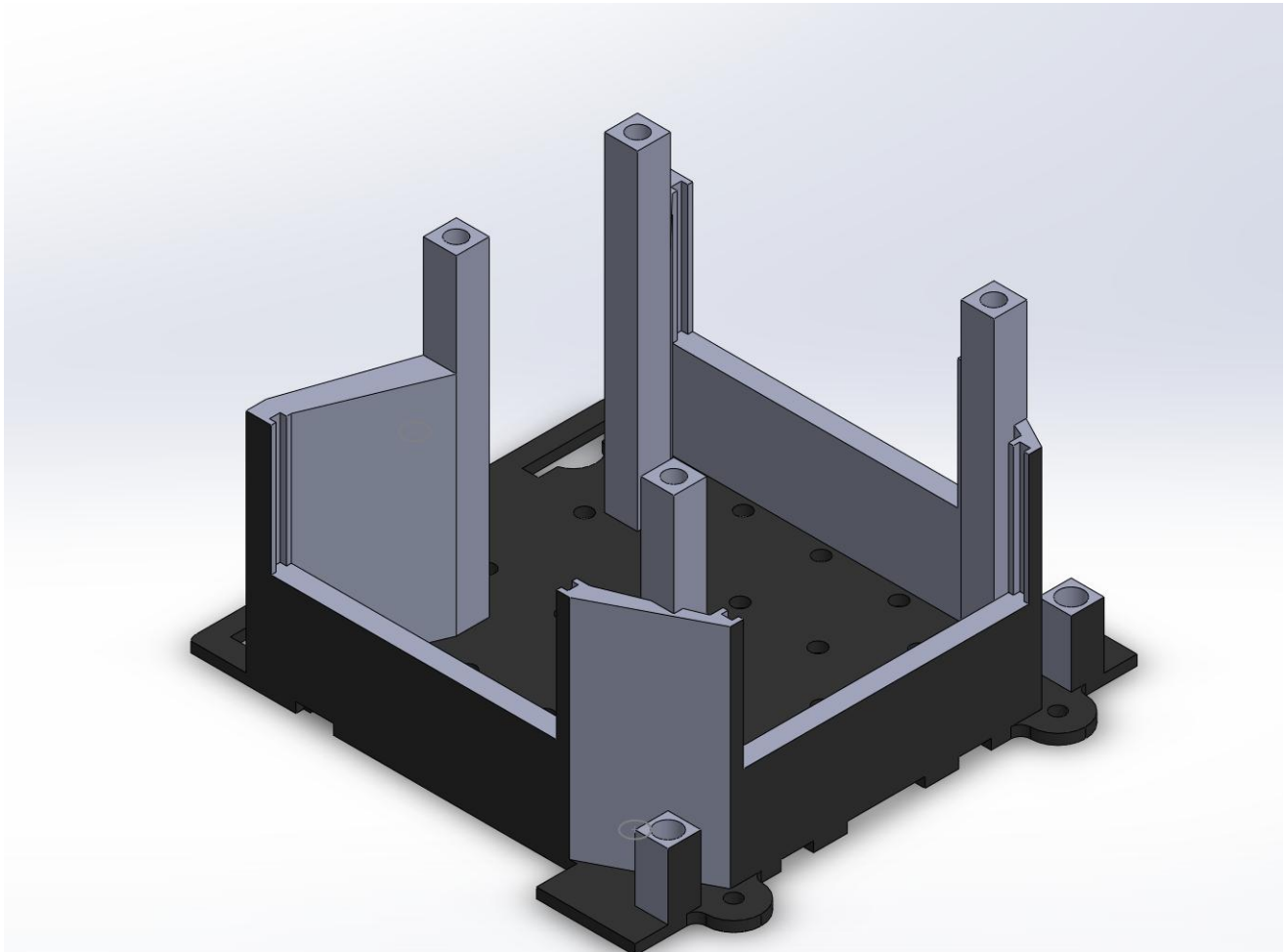
Figure 1. System architecture block diagram showing robot inputs, controller, outputs, and power distribution.

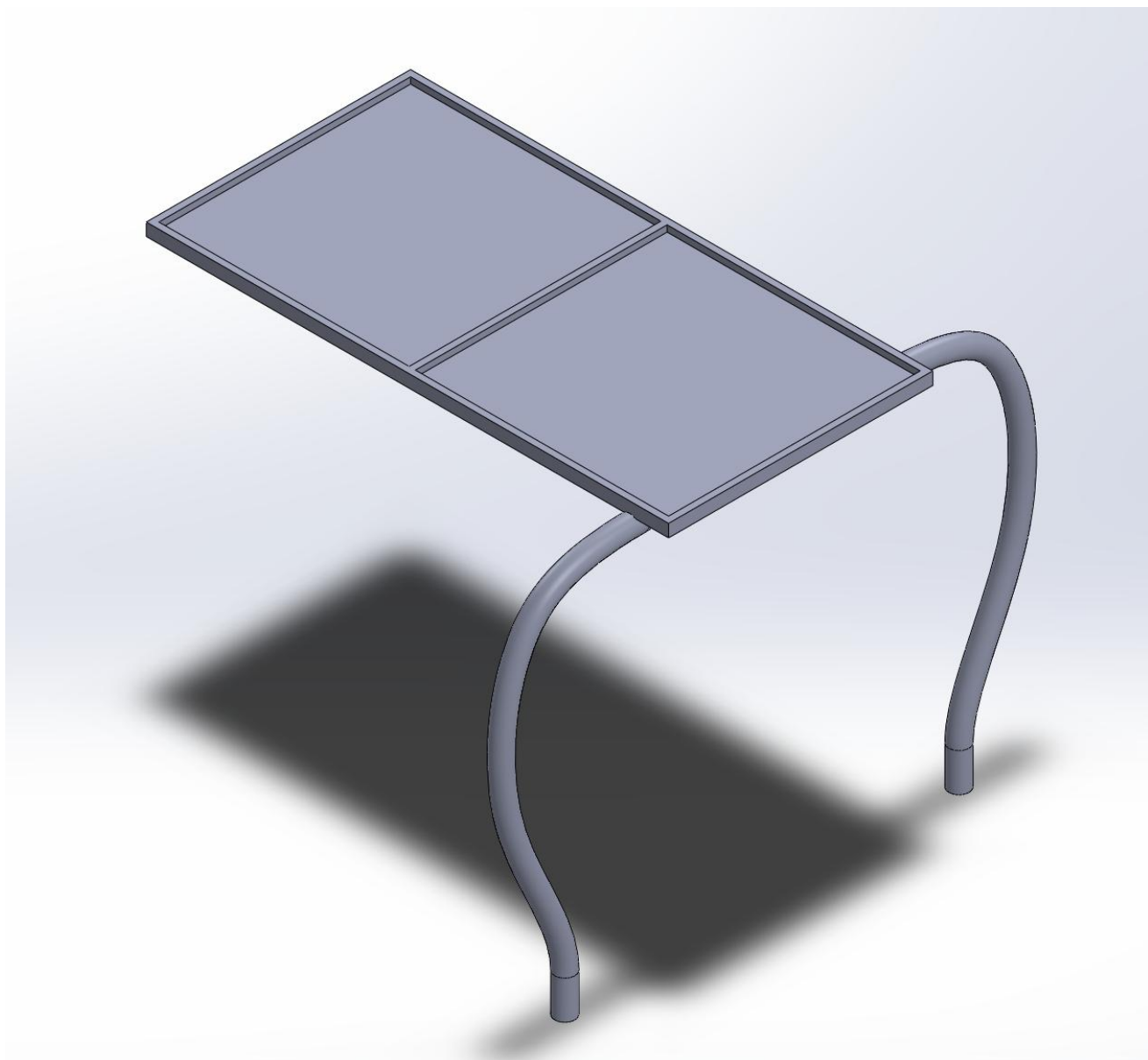
5.2 Mechanical Design

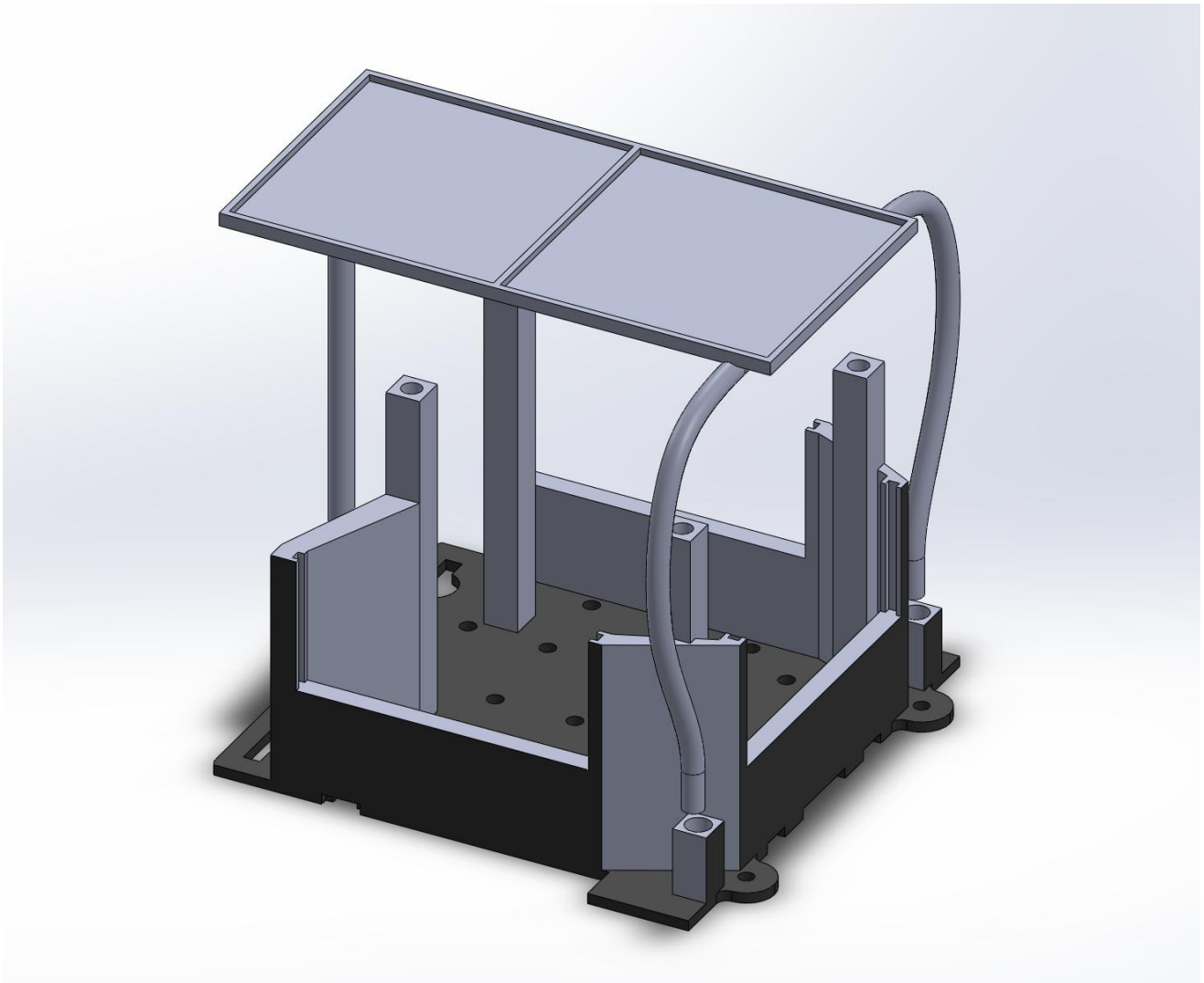
The mechanical design supports the electronic components while keeping the robot stable during movement. The robot body uses the provided chassis as the base and adds a 3D-printed bracket for two ArUco markers, mounts for ultrasonic sensors, and a protected area for the OLED display and wiring. The ArUco bracket is designed to keep the markers flat, visible, and below the 6.5 in height limit. The bracket is secured mechanically instead of with glue or tape.

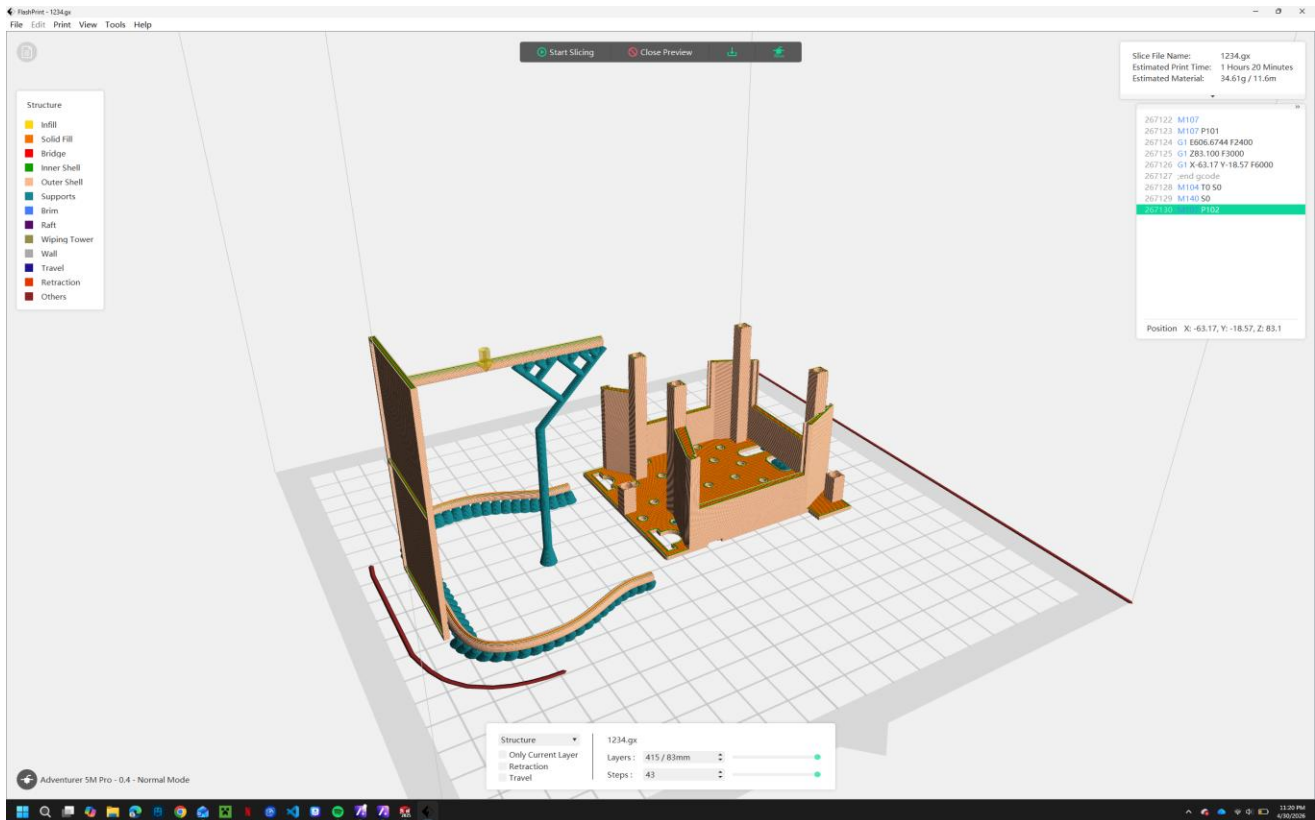
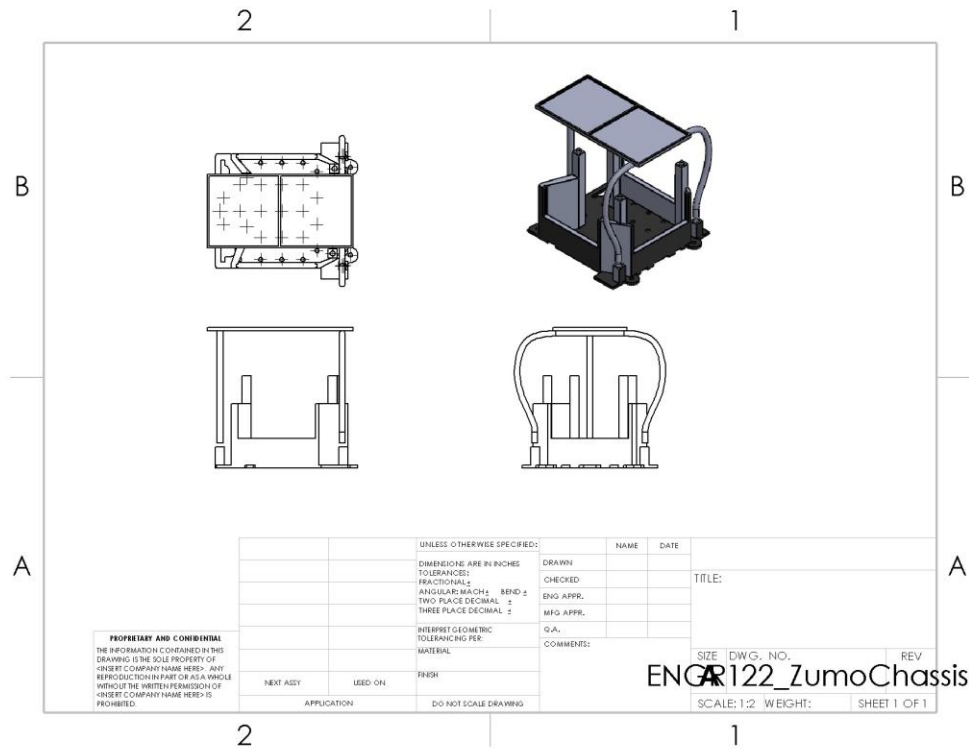
The selected ArUco bracket uses a low bridge design. This design was chosen because it provides camera visibility without raising the center of gravity too much. A stable bracket is important because the center point of the two markers is used as the robot location in the tracking system. Any vibration or marker tilt could reduce tracking accuracy and cause path-planning errors.

The ultrasonic sensors are placed so that the robot can detect objects in front of it and on both sides. This placement helps the software determine whether to move forward, slow down, or turn away from an obstacle. The mounts are positioned to reduce false readings from the robot body and keep the sensor faces exposed to the arena.









(Estimated print time – 80 minutes)

5.3 Electrical Circuit and Wiring Design

The electrical design uses a no-breadboard final circuit. Power is distributed through a power bus that branches to the WeMos D1 R1, OLED display, ultrasonic sensors, and motor controller board. A slide switch is placed on the positive battery line so the entire robot can be powered off safely. This reduces the risk of unexpected motor or sensor behavior when the robot is not being tested.

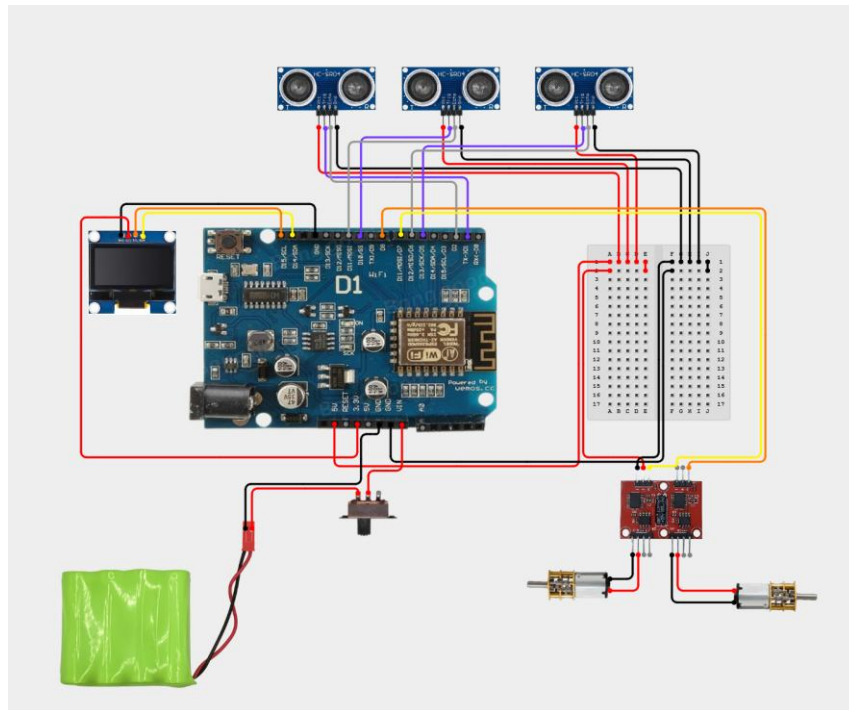


Figure 2. Wiring concept showing main power line, slide switch, power bus, controller, sensors, OLED, and motor driver.

The color coding shown is red for positive voltage connections, including 5V, 3.3V, VCC, VIN, and battery positive. Black is used for ground and negative connections. White and purple were used for the Echo and Trigger pins for the ultrasonic sensors, while orange and yellow were used for the OLED display and the motor controller.

Device	Connection Type	Signal/Power	Purpose
OLED display	I2C	SDA, SCL, VCC, GND	Displays sensor data and robot angle
Ultrasonic sensors	GPIO	Trigger, Echo, VCC, GND	Measures obstacle/wall distance
Motor Controller	PWM/GPIO	Speed PWM and direction pins	Controls left and right motors
Motor	DC (from SMC)	VCC, GND	Drives the track treads, moving the robot
Switch	VCC	Positive battery line	Cuts off power to the board

5.4 Software Design and Coding

The software design builds on the baseline provided MQTT code and adds the team-developed path-planning and obstacle-avoidance algorithm. The program begins by initializing Wi-Fi, MQTT, ultrasonic sensors, motors, and the OLED display. The robot then waits two seconds at the start before beginning the target sequence. During each control loop, the robot reads its current x-y position, calculates the distance and direction to the active target, checks ultrasonic sensor distances, and chooses motor commands.

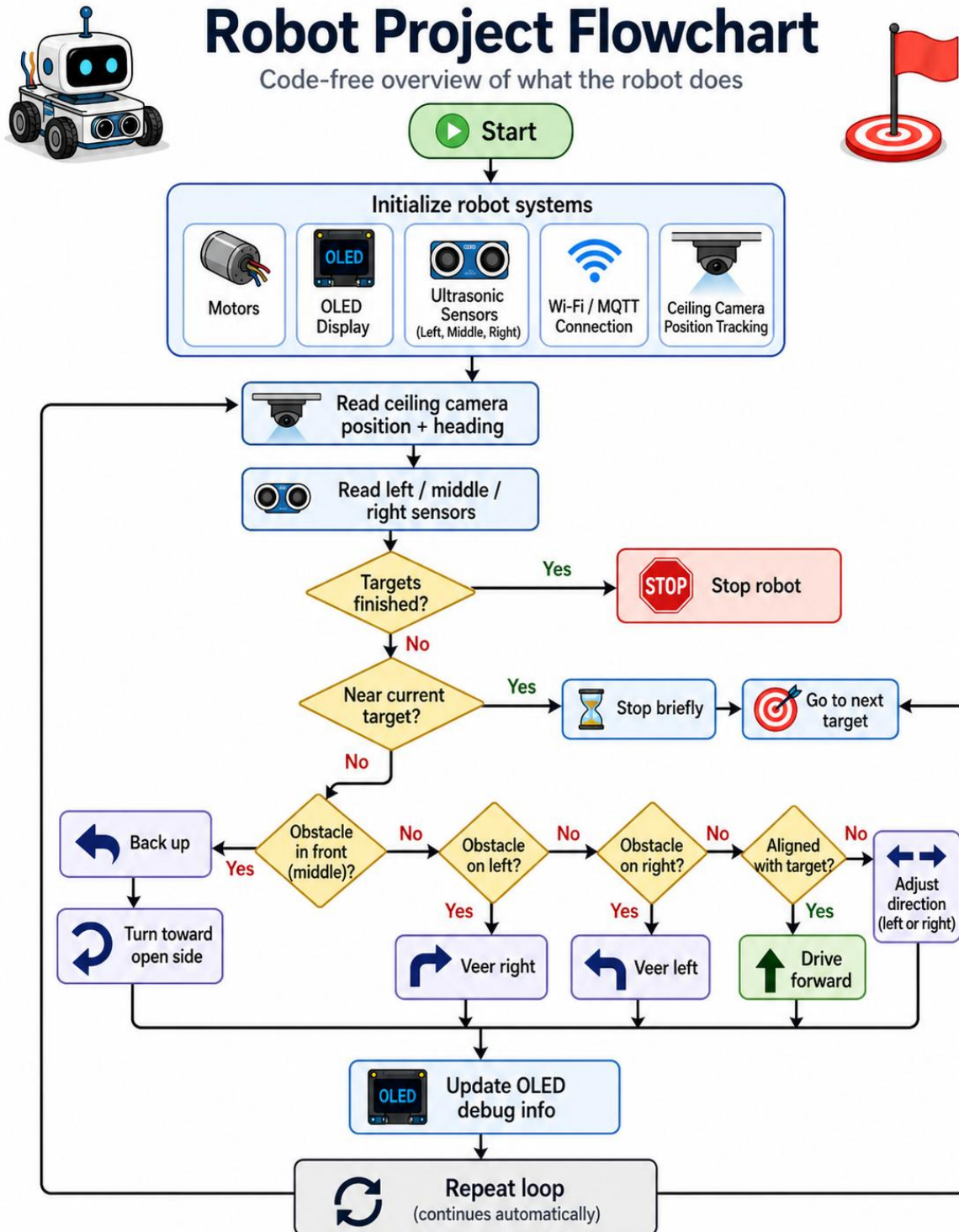


Figure 3. Software flowchart for target reaching and collision avoidance.

The robot uses a WiFi and MQTT network to receive real-time position data from the arena camera system. In the code, the robot connects to the North Arena WiFi network using the assigned router credentials, then communicates with the MQTT broker at 192.168.0.2 on port 1883. The MQTT topic is set to EAS011_North, which allows the robot to receive its live x, y, and heading angle data from the camera system.

This network connection is essential because the robot does not determine its position on its own. Instead, it relies on MQTT messages from the camera system to continuously update `x_robot`, `y_robot`, and `z_ang_robot`. The program then uses this information to calculate the distance and angle to the next target, allowing the robot to make steering decisions and correct its path while moving through the arena

The target-reaching logic compares the current robot coordinates to the active target coordinates. If the error distance is greater than 10 cm, the robot continues moving toward the target. If the robot enters the target radius, the motors stop for 2 seconds, and the program advances to the next target. After all four targets are reached, the robot stops.

The obstacle-avoidance logic uses ultrasonic readings to determine whether an object is too close to the robot. If the front sensor detects an obstacle too close, the robot turns 90° to the side that has most free space, determined by the ultrasonic sensors on the left and right. Additionally the robot This logic allows the robot to correct its path without crashing into obstacles or arena walls.

To path find toward each target, the robot uses its current coordinates and the target's coordinates to calculate the angle of a straight-line path to the target. It then compares this desired angle with the robot's current heading angle, which is received through the MQTT network. By subtracting the two angles, the robot finds the angle error. The angle error is then mapped to stay within the range of -180° to 180°, which allows the robot to determine the shortest direction to turn. If the angle error is greater than about $\pm 15^\circ$, the robot recognizes that it is not facing the correct general direction. It then corrects its path by veering left or right depending on which direction reduces the angle error. The size of the correction is proportional to the angle error. This means that a larger angle error causes the robot to swerve more aggressively, while a smaller angle error causes a smaller correction. This allows the robot to gradually steer itself back toward the straight-line path to the target.

The constants came from testing different slopes in Desmos. Dividing by a smaller number creates a steeper line, meaning the motor value changes faster as the angle error increases. Dividing by a larger number creates a slower, more gradual change. We used Desmos to make sure the motor values stayed in a reasonable range instead of becoming too extreme. This lets the robot correct its path without spinning too sharply or losing control.

Here, the constants are not all the same because the two motors did not behave exactly equally. One motor needed a stronger adjustment than the other to make the robot turn smoothly. Desmos made it easier to compare these lines and choose constants that gave a smooth increase in correction as the angle error increased.

It was calculated using this Desmos: <https://www.desmos.com/calculator/rkbljcicsi>

6. Assembly and Prototyping

During the assembly and prototyping stage, our first design failed because we found that the print was not strong enough, as it was made with only 20% infill, so we increased the infill to 80% to make the structure much sturdier and better able to handle use. In addition, the original frame was not stable enough overall, so we added four supports around each corner to reinforce it and improve its durability. Finally, we made the three pegs that hold the ArUco markers longer so they could fit properly into the three holes around the edges, which helped the final design come together more securely and functionally. We tried to keep the model simple by placing the sensors at 90-degree angles, but we did not account for the blind spots that this arrangement created. As a result, the design could not provide the coverage and reliability we needed. However, the autonomous robot was still able to navigate the course, making our design a success overall.

Item	Qty	Purpose	Source/Notes	Status
Robot chassis/base	1	Main structure	ENGR 122 kit	Installed
WeMos D1 R1	1	Main controller	ENGR 122 kit	Installed
Motor controller board	1	Motor control	ENGR 122 kit	Installed
DC motors	2	Drive system	ENGR 122 kit	Installed
HC-SR04 ultrasonic sensors	3	Obstacle detection	ENGR 122 kit	Installed
OLED display	1	Debug display	ENGR 122 kit	Installed
ArUco markers	2	Camera tracking	Printed markers	Installed
3D-printed marker bracket	1	Marker support	Student-designed part	Installed
Slide switch	1	Power cutoff	ENGR 122 kit	Installed
Battery pack	1	Power source	ENGR 122 kit	Installed
Power bus wires/PCB	1 set	Power distribution	Custom wiring	Installed
Heat-shrink tubing	As needed	Insulation	Lab supplies	Installed

Table 1. Bill of Materials for the autonomous robot system.

7. Performance Reporting and Project Management

The design was tested in stages before the final competition. The first tests verified individual devices. The OLED was checked by displaying test text and coordinate values. The ultrasonic sensors were tested by placing objects at known distances and comparing the measured readings. The motor driver was tested by running left and right motors forward and backward. The MQTT connection was tested by confirming that live x-y coordinate data appeared on the serial monitor or OLED display.

After subsystem testing, the robot was tested in the arena. The team first used simple movement commands to confirm that the robot could drive straight and turn reliably. Next, the robot was tested with one target coordinate. After the single-target test worked, the program was expanded to multiple targets. Finally, obstacles were added and the team tuned the obstacle safety threshold and turning behavior.

We were able to reach all 4 targets in the allotted time. Well under 3 minutes made up for the 3 hit walls our robot had. We overlooked the large blind spots our robot had which made us have a hard time with the path-finding algorithm. We needed to find a buffer zone that was big enough for the robot to not hit any walls while also not constantly going in circles. Overall, we were very happy with the outcome.

Test	Goal	Result	Issue Found	Action Taken
OLED display test	Show current status and target	[Pass/Fail]	[Issue]	[Fix]
MQTT coordinate test	Receive x-y location	[Pass/Fail]	[Issue]	[Fix]
Ultrasonic distance test	Measure obstacles	[Pass/Fail]	[Issue]	[Fix]

	reliably			
Motor direction test	Correct left/right motion	[Pass/Fail]	[Issue]	[Fix]
Single-target test	Reach one target within 10 cm	[Pass/Fail]	[Issue]	[Fix]
Four-target sequence	Reach all targets in order	[Pass/Fail]	[Issue]	[Fix]
Collision avoidance test	Avoid arena obstacles	[Pass/Fail]	[Issue]	[Fix]
Competition trial	Complete mission within time limit	[Score/Time]	[Issue]	[Fix]

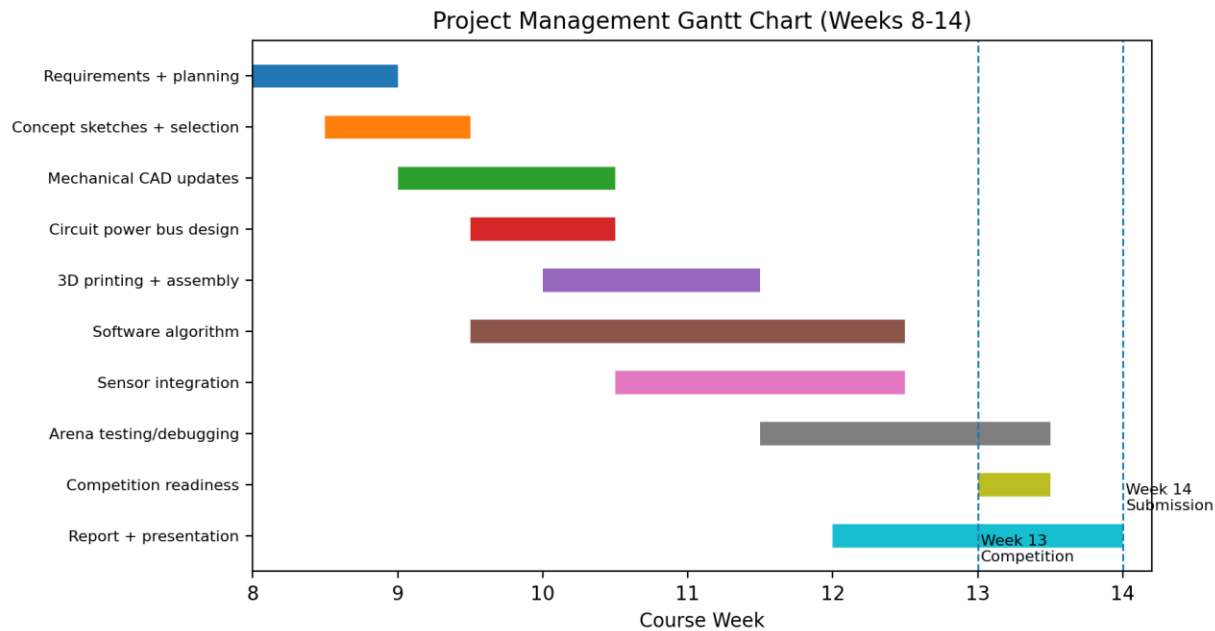
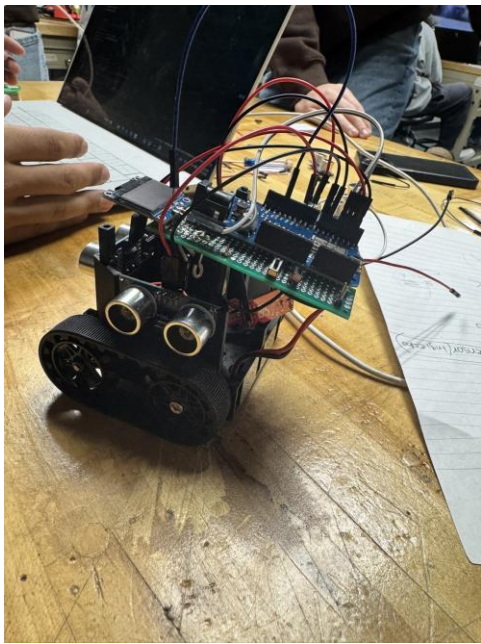


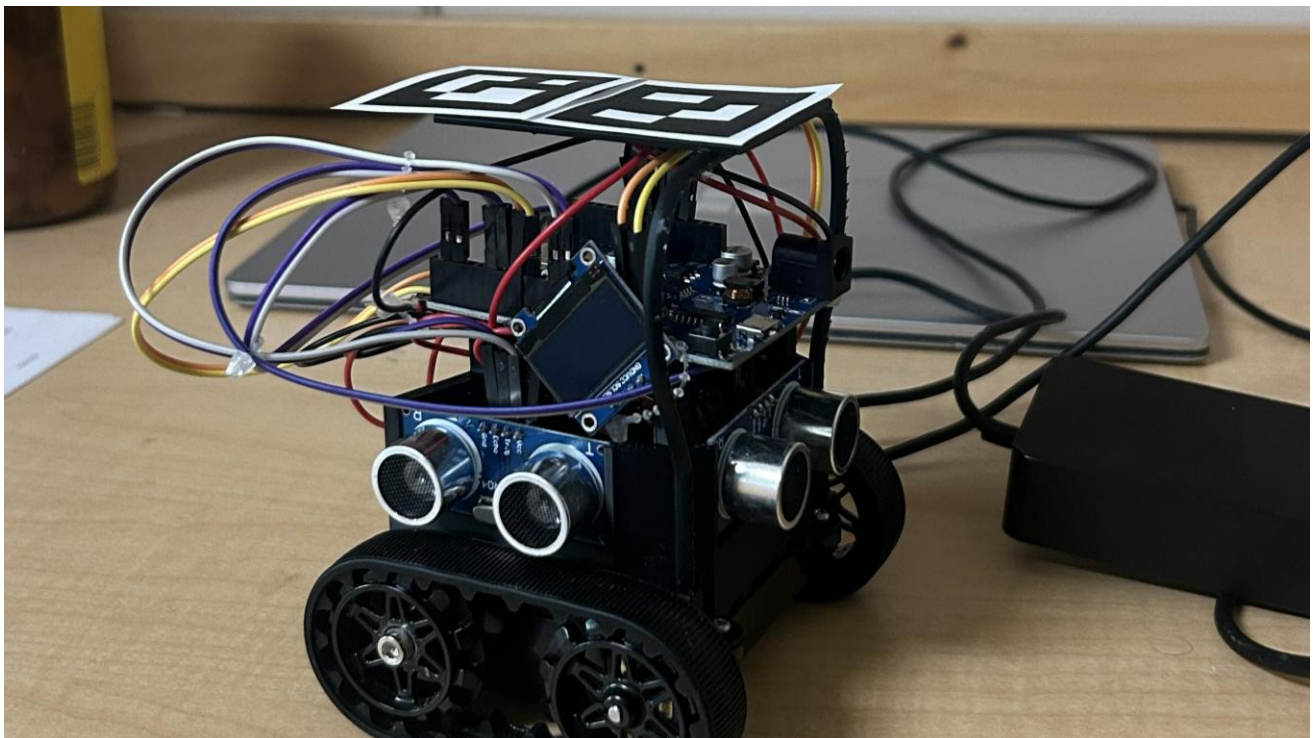
Figure 4. Gantt chart for project planning from Week 8 to Week 14.

The Gantt chart helped divide work across the six-week project window. Mechanical design and production were scheduled early because the robot could not be tested until the physical mounts and wiring were complete. Software development and testing were given the largest amount of time because the project required the team to develop its own path-planning and obstacle-avoidance logic. The Week 13 competition was treated as the major milestone, while Week 14 was reserved for the final presentation and submission package.

That was until our board was accidentally fired on week 12. We had a custom circuit seen in the picture below but at the last minute had to switch board types which caused us to have to use a breadboard due to time constants. This decision was given the ok by Professor Joo.



Final Design:



8. Conclusion and Recommendation

The autonomous robot design satisfies the main project goal by combining a stable mechanical body, organized electrical wiring, live coordinate tracking, ultrasonic obstacle detection, and software-based target reaching. The final design is intended to guide visitors through the arena by moving toward client-provided

target coordinates while avoiding obstacles. The use of the ArUco marker bracket supports reliable camera tracking, power bus improves reliability compared with a temporary prototype circuit.

The most important lesson from the project is that successful autonomous movement depends on the entire system working together. Mechanical vibration can affect marker tracking, wiring quality can affect sensor reliability, sensor placement can affect obstacle detection, and software tuning can affect whether the robot reaches targets smoothly. Because of this, each subsystem had to be tested individually and then tested again as part of the full robot.

Appendix A: Full Arduino Sketch

```
/*=====
=====
Ver.1 - 03/02/2026 SES IDEAs Program
2026S ENGR122 Sample Code for Week 7 Activity & Final Project
Camera Sensor-Based Position Feedback using MQTT Network
*
This sketch is provided as a baseline for the Week 7 lab activity and Final Project
Week7 Activity:
-Based on this baseline code, students must complete the program to display the robot's position
on the OLED display.
Final Project:
-Using the ultrasonic sensor, students must design an Obstacle Avoidance logic.
-Using camera-sensor data received through the MQTT network, students must implement a Path-
Planning algorithm.
-Students must integrate these algorithms to control the two motors so that the robot can reach
designated targets.
-The OLED display should be used for debugging purposes.
*
IMPORTANT ACADEMIC INTEGRITY NOTICE-----
Sample codes provided each semester may include hidden identifiers to detect unauthorized copying.
Copying, sharing, posting, or uploading this Arduino Sketch code to any online platform or any
other repository service
(ex. github...) is strictly prohibited and will be reported as a violation of the Stevens Honor
System.
=====
=====*/
//(NOTE!) Include all required libraries here
#include "MQTT.h"
#include <Wire.h>
#include <SSD1306Wire.h>
#include <Ultrasonic.h>
#include <Servo.h>
#include <math.h>

//(NOTE!) You may create any additional objects needed for OLED, Sensors, or Motors here

#define motorLpin D7
#define motorRpin D8

Servo motorL;
Servo motorR;

Ultrasonic ultrasonic_middle(D5, D6);
Ultrasonic ultrasonic_right(D10, D11);
Ultrasonic ultrasonic_left(D1, D2);

SSD1306Wire display(0x3C, D14, D15);
//target coordinates w/ 4 midpoints placed in
const int x_targets[] = {691, 720, 1541, 2162, 2125, 917, 111}; //Sample x-coordinates for targets
const int y_targets[] = {676, 145, 98, 118, 553, 610, 114}; //Sample y-coordinates for targets

int target_index = 0;

//MQTT Setting (Common configuration used across both South Arena and North Arena systems)
const char* mqtt_server = "192.168.0.2";
const char* mqtt_username = "user";
const char* mqtt_password = "Stevens1870";
const int mqtt_port = 1883;

//MQTT Topic
const String topic = "EAS011_North"; //North Arena
//const String topic = "EAS011_South"; //South Arena
```

```

//const String topic = "SmartArena";

//WiFi Setting
const char* ssid      = "TP-Link_8FFD";      //North Arena Check the router on the TA desk.
const char* password   = "68287078";        //North Arena Check the router on the TA desk.
//const char* ssid      = "TP-Link_9402";      //South Arena Check the router on the TA desk.
//const char* password = "77556578";        //South Arena Check the router on the TA desk.
//const char* ssid = "TP-Link_7310";
//const char* password = "74952359";

float x_robot, y_robot, z_ang_robot;

//(NOTE!) You may add additional variables here as needed, but carefully consider variable scope
(global vs. local)
//initilize variables for ultrasonic sensor readings
int distance_middle;
int distance_right;
int distance_left;
float angle_error;

const char* robot_state = "INIT";

void setup() {
  Wire.begin();
  delay(25);
  wifi_mqtt_init();
  mqtt_clean();
  delay(500);

  display.init();
  display.flipScreenVertically();

  motorL.attach(motorLpin);
  motorR.attach(motorRpin);

  motorL.write(90);
  motorR.write(90);
  delay(2000);

}
//move the rebot forward
void forward(){
  robot_state = "FORWARD";
  motorL.write(73);
  motorR.write(75);
}
//stop the robot
void stop(){
  robot_state = "STOP";
  motorL.write(90);
  motorR.write(90);
}
//move slightly left depending on the distance to the target
void veer_left(int mag, int dist){
  robot_state = "SWERVE_L";
  if(mag==0){
    motorL.write(68+dist);
    motorR.write(65-dist);
  } else {
    motorL.write(68+(mag/7.5));
    motorR.write(65-(mag/11));
  }
}

```

```

//move slightly right depending on the distance to the target
void veer_right(int mag, int dist){
robot_state = "SWERVE_R";
if(mag==0){
    motorL.write(58-dist);
    motorR.write(68+dist);
} else {
    motorL.write(58-(mag/21));
    motorR.write(68+(mag/7.5));
}
}

//turn 90 degrees left
void small_turn_left(){
robot_state = "STURN_L";
motorL.write(115);
motorR.write(70);
delay(150);
}

//turn 90 degrees right
void small_turn_right(){
robot_state = "STURN_R";
motorL.write(70);
motorR.write(115);
delay(150);
}

//reverse the robot
void reverse(){
robot_state = "REVERSE";
motorL.write(123);
motorR.write(120);
}

}

void loop() {

mqtt_rebound();
client.loop();

//read ultrasonic sensors
distance_middle = ultrasonic_middle.read(CM);
delay(10);
distance_right = ultrasonic_right.read(CM);
delay(10);
distance_left = ultrasonic_left.read(CM);
delay(10);

//if all targets reached stop robot
if(target_index < 7){
    //calculate distance to target
    float dx = x_targets[target_index] - x_robot;
    float dy = y_targets[target_index] - y_robot;
    float distance_to_target = sqrt(dx*dx + dy*dy);

    //if too close to target, reverse shortly and turn
    if(distance_to_target < 100){
        stop();
        delay(2000);
        target_index++;
    }
}
}

```

```

    else {
        if(distance_middle < 12){
            reverse();

            if(distance_left > distance_right){
                small_turn_left();
            }
            else{
                small_turn_right();
            }
        }
    }
    else if(distance_left < 15){
        veer_right(0, distance_left);
    }
    else if(distance_right < 15){
        veer_left(0, distance_right);
    }
    else{
        //calculate the angle to the target
        float desired_angle = atan2(dy, dx) * 180 / PI;
        angle_error = desired_angle - z_ang_robot;

        //map the angle to [-180,180]
        if(angle_error > 180) angle_error -= 360;
        if(angle_error < -180) angle_error += 360;

        //correct to the right angle path
        if(angle_error > 10){
            veer_left(abs(angle_error), 0);
        }
        else if(angle_error < -10){
            veer_right(abs(angle_error), 0);
        }
        else{
            forward();
        }
    }
}
}
}
}
else{
    stop();
    delay(2000);
    return;
}

//print to OLED display --> sensor data and Angle error
display.setFont(ArialMT_Plain_10);
display.flipScreenVertically();
display.clear();

display.drawString(0, 0, "Dist Mid: " + String(distance_middle));
display.drawString(0, 15, "Left: " + String(distance_left));
display.drawString(0, 30, "Right: " + String(distance_right));
display.drawString(0, 45, "Angle Error: " + String(angle_error));
display.display();

mqtt_clean();
}

```